

Rozbijam Publiczny Profil Twórcy (4.1.2) na elementy atomowe + checklistę z opisem każdego bloku.

4.1.2 Publiczny Profil Twórcy (Widok dla fana/odwiedzającego)

Checklist (atomy funkcjonalne i wizualne):

A. Banner (Cover Image)

Obrazek w tle (opcjonalnie personalizowany przez twórcę)

Minimalne wymiary / responsywność / lazy loading

Domyślna grafika, gdy twórca nie ustawił własnej

B. Avatar (Profilowe)

Okrągłe zdjęcie twórcy na tle banneru (złota ramka, efekt podświetlenia)

Placeholder/avatar default jeśli brak zdjęcia

Przycisk „zmień” tylko dla właściciela (nie dla fana)

C. Dane Twórcy

Wyświetlana nazwa (displayName, pogrubiona, Montserrat)

Alias/username (np. @nazwa, mniejszą czcionką pod nazwą)

Krótki opis/bio twórcy (max X znaków, line-clamp-3)

Ikony social media/linki (YouTube, Twitch, X, Instagram – tylko podlinkowane przez twórcę)

D. Cel Zbiórki (Goal Widget)

Pasek postępu z kwotą docelową i aktualną

Tekstowy opis celu (np. „Nowy mikrofon”, opcjonalnie data zakończenia)

Prezentacja % postępu w pasku lub liczbowo

E. Przycisk CTA „Wesprzyj”

Złoty przycisk CTA („Wyślij Napiwek”/„Support”) – widoczny, duży

Otwiera modal/formularz napiwku

Ewentualnie disabled, jeśli profil nieaktywny

F. Adres do wpłaty (krypto) / QR

Ikona/tooltip „Adres do wpłaty”

Po kliknięciu modal z adresem portfela Circle twórcy + kod QR do wpłaty USDC

Przycisk „Kopiuje adres”

G. Historia Wsparcia

Lista ostatnich napiwków: nick fana (lub „Anonimowy”), kwota, data/godzina, opcjonalnie wiadomość

Dobrze widoczne awatary/nicki dla społecznego dowodu

H. Komentarze / Tablica Wsparcia (opcjonalnie)

Krótkie wiadomości od fanów po wsparciu (max 1-2 linijki)

Możliwość filtrowania (ostatnie/wszystkie)

I. Dodatki/Integracje

Przycisk „Udostępnij” (share – social, link, QR)

Liczba obserwujących/wspierających (jeśli zaimplementowane)

Przyciski „Obserwuj” lub „Do ulubionych” (opcjonalnie)

J. Responsywność i UX

Wyśrodkowanie najważniejszych informacji na mobile

CTA i pole do wsparcia zawsze sticky/przyklejone do dołu na mobile

Animacja pojawiania się nowego napiwku na liście (np. flash złotem)

Checklist wdrożeniowa Publicznego Profilu Twórcy

1. ☐ Banner (z obsługą domyślnego i uploadu)
2. ☐ Avatar (z efektem, domyślny + upload)
3. ☐ Wyświetlanie danych twórcy (nazwa, alias, bio, linki)
4. ☐ Cel zbiórki (widget postępu, opis celu)
5. ☐ Główny przycisk CTA „Wesprzyj” (modal/formularz napiwku)
6. ☐ Alternatywna opcja – adres do wpłaty + QR (modal)
7. ☐ Historia wsparcia (lista ostatnich napiwków)
8. ☐ Komentarze/tablica (opcjonalnie)
9. ☐ Integracje (udostępnianie, licznik followersów)
10. ☐ Responsywność, sticky CTA na mobile, animacje
11. ☐ Edge-case’y: profil nieaktywny, brak historii, anonimowy twórca

Podsumowanie: Każdy punkt możesz traktować jako osobny „task” dla dev/designerów. Chcesz, żebym rozpisał detale modalów lub każdy blok w formie wireframe/diagramu? Czy przechodzimy do następnej strony/podstrony?

```

'use client';

import { useState } from 'react';
import Image from 'next/image';

// Placeholder dane – tu wstawisz fetch/getServerSideProps
const creator = {
  bannerUrl: '/assets/cover.png',
  avatarUrl: '/assets/avatar.png',
  displayName: 'Adam Duda',
  username: 'AdamDuda',
  bio: 'Fan paleo-kontaktu, buduje TipJar+. Web3 & AI pro user.',
  socials: [
    { type: 'youtube', url: 'https://youtube.com/@AdamDuda' },
    { type: 'twitch', url: 'https://twitch.tv/AdamDuda' },
  ],
  goal: {
    title: 'Nowy mikrofon',
    current: 57,
    target: 120,
  },
  circleWallet: '0x1234...abcd',
  history: [
    { user: 'CryptoFan', amount: 5, date: '2025-06-20', msg: 'Super stream!', avatar: '/assets/u1.png' },
    { user: 'Anonimowy', amount: 2, date: '2025-06-18', msg: '', avatar: '/assets/anon.png' },
  ]
};

export default function CreatorProfilePage() {
  const [showTip, setShowTip] = useState(false);
  const [showQR, setShowQR] = useState(false);

  // --- helpers
  const percent = Math.min(100, Math.round((creator.goal.current / creator.goal.target) * 100));

  return (
    <div className="min-h-screen bg-[#003737] text-white">
      { /* Banner */ }
      <div className="relative w-full h-[240px] md:h-[320px] bg-slate-800 overflow-hidden">
        <Image
          src={creator.bannerUrl}
          alt="Banner twórcy"
          fill
          className="object-cover"
          priority

```

```

/>
{ /* Avatar */
  <div className="absolute left-1/2 -bottom-16 md:-bottom-20 transform -translate-x-1/2
z-10">
    <div className="w-32 h-32 md:w-40 md:h-40 rounded-full border-4
border-[#FFD700] shadow-lg bg-[#003737] flex items-center justify-center overflow-hidden
animate-pulse-slow">
      <Image
        src={creator.avatarUrl}
        alt="Avatar"
        width={160}
        height={160}
        className="object-cover w-full h-full"
      />
    </div>
  </div>
</div>

<main className="pt-24 md:pt-28 max-w-3xl mx-auto px-2">
  { /* Dane twórcy */
    <div className="flex flex-col items-center gap-2">
      <span className="text-3xl font-montserrat font-bold">{creator.displayName}</span>
      <span className="text-[#FFD700] font-mono
text-base">@{creator.username}</span>
      <span className="text-center text-slate-200 mt-2 max-w-2xl">{creator.bio}</span>
      { /* Socials */
        <div className="flex gap-3 mt-2">
          {creator.socials.map(s => (
            <a
              key={s.type}
              href={s.url}
              target="_blank"
              rel="noopener noreferrer"
              className="hover:text-[#FFD700] transition"
              title={s.type}
            >
              {s.type === 'youtube' && (
                <svg width="26" height="26" fill="currentColor"><path d="M24 6.3a2.86 2.86 0 0
0-2-2C20 4 13 4 13 4s-7 0-9 .3a2.86 2.86 0 0 0-2 2A29.4 29.4 0 0 0 1 13a29.4 29.4 0 0 0 1
6.7 2.86 2.86 0 0 0 2 2C6 22 13 22 13 22s7 0 9-.3a2.86 2.86 0 0 0 2-2A29.4 29.4 0 0 0 25
13a29.4 29.4 0 0 0-1-6.7zM10 17.6V8.4L17.5 13z"/></svg>
              )}
              {s.type === 'twitch' && (
                <svg width="26" height="26" fill="currentColor"><path d="M3 0L0
5.2v18.3h5.2V26h3.5l3.5-3.5h5.2l6.3-6.2V0H3zm20.7 14.5l-4.2 4.1h-5.2l-3.5
3.5v-3.5H3.5V1.7h20.2zM18 8h-1.7v5.2H18zm-5.2 0H11v5.2h1.7z"/></svg>
              )}
            </a>
          )}
        </div>
      )}
    </div>
  )}

```

```

    ))}
  </div>
</div>

{/* Cel Zbiórki */}
<div className="mt-8 flex flex-col items-center">
  <div className="flex items-center gap-3">
    <span className="font-semibold text-xl">{creator.goal.title}</span>
    <span className="bg-[#FFD700] text-[#003737] font-bold px-2
rounded-lg">{percent}%</span>
  </div>
  <div className="w-full bg-[#008080]/30 rounded-full h-5 mt-2 relative
overflow-hidden">
    <div className="bg-[#FFD700] h-full rounded-full transition-all" style={{ width:
`${percent}%` }} />
    <span className="absolute inset-0 flex justify-center items-center text-xs
font-semibold text-white">{creator.goal.current} / {creator.goal.target} USDC</span>
  </div>
</div>

{/* CTA + QR/Address */}
<div className="flex flex-col md:flex-row items-center justify-center gap-4 mt-8">
  <button
    className="font-bold bg-[#FFD700] text-[#003737] px-8 py-3 rounded-2xl text-lg
shadow-lg hover:scale-105 hover:shadow-yellow-400/30 transition"
    onClick={() => setShowTip(true)}
  >
    Wyślij Napiwek
  </button>
  <button
    className="flex items-center gap-2 text-[#FFD700] font-semibold hover:underline"
    onClick={() => setShowQR(true)}
  >
    <svg width="22" height="22" fill="none" stroke="#FFD700" strokeWidth="2"><rect
x="3" y="3" width="6" height="6" rx="1.5"/><rect x="13" y="3" width="6" height="6"
rx="1.5"/><rect x="3" y="13" width="6" height="6" rx="1.5"/><rect x="13" y="13" width="6"
height="6" rx="1.5"/></svg>
    Adres do wpłaty
  </button>
</div>

{/* Historia wsparcia */}
<div className="mt-12">
  <div className="font-bold text-lg mb-4">Ostatnie napiwki</div>
  <div className="grid gap-3">
    {creator.history.map((t, i) => (
      <div key={i} className="flex items-center gap-3 bg-white bg-opacity-5 p-3
rounded-xl">

```

```

<Image src={t.avatar} alt="" width={36} height={36} className="rounded-full" />
<div className="flex-1">
  <span className="font-semibold">{t.user}</span>
  <span className="ml-2 text-sm text-[#FFD700] font-mono">{t.amount}
USDC</span>
  <div className="text-xs text-slate-400">{t.date}</div>
  {t.msg && <div className="italic text-slate-200">{t.msg}</div>}
</div>
</div>
)}}
{!creator.history.length && (
  <div className="text-slate-400 italic">Jeszcze nikt nie wsparł tego twórcy. Bądź
pierwszy!</div>
)}
</div>
</div>
</main>

```

```

{/* Modal: Wysyłanie napiwku */}
{showTip && (
  <div className="fixed inset-0 z-40 flex items-center justify-center bg-black
bg-opacity-70">
    <div className="bg-[#003737] p-8 rounded-2xl shadow-2xl max-w-sm w-full
relative">
      <button className="absolute top-2 right-2 text-2xl text-[#FFD700] font-bold"
onClick={() => setShowTip(false)}>×</button>
      <div className="font-bold text-lg mb-4 text-center">Wyślij napiwek twórcy</div>
      {/* Tu wrzuć logikę napiwku */}
      <div className="flex flex-col gap-2">
        <input type="number" placeholder="Kwota USDC" className="p-2 rounded
bg-slate-800 text-white"/>
        <textarea placeholder="Wiadomość dla twórcy (opcjonalnie)" className="p-2
rounded bg-slate-800 text-white" />
        <button className="bg-[#FFD700] text-[#003737] font-bold py-2 rounded-xl
mt-2">Wyślij</button>
      </div>
    </div>
  </div>
)}

```

```

{/* Modal: Adres do wpłaty */}
{showQR && (
  <div className="fixed inset-0 z-40 flex items-center justify-center bg-black
bg-opacity-70">
    <div className="bg-[#003737] p-8 rounded-2xl shadow-2xl max-w-sm w-full
relative">
      <button className="absolute top-2 right-2 text-2xl text-[#FFD700] font-bold"
onClick={() => setShowQR(false)}>×</button>

```

```

<div className="font-bold text-lg mb-4 text-center">Adres portfela twórcy</div>
{/* Tu generuj QR kod np. z react-qr-code */}
<div className="flex flex-col items-center gap-2">
  <div className="w-32 h-32 bg-white flex items-center justify-center rounded-lg">
    <span className="text-xs text-black text-center p-4">[QR dla
{creator.circleWallet}]</span>
  </div>
  <div className="bg-slate-900 text-white px-3 py-1 rounded mt-2 select-all
text-center text-sm">{creator.circleWallet}</div>
  <button className="text-[#FFD700] mt-2" onClick={() =>
{navigator.clipboard.writeText(creator.circleWallet)}}>Kopiuj adres</button>
</div>
</div>
</div>
)}
</div>
);
}

```

Podsumowanie:

Cały szkielet z każdym blokiem, bez pustych miejsc.

Wszystko gotowe pod integrację z backendem/API, QR i logiką.

Styl: 100% Tailwind, układ pod mobile i desktop.

UX: sticky CTA, animacje, dostępność.